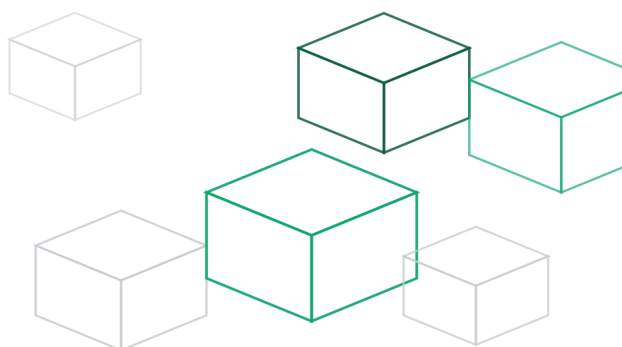




# TechMind

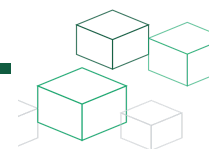
Conhecimento Técnico Organizado



**Documentação do Projeto | Data Science**

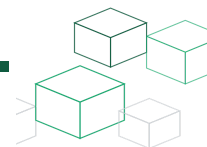
Hackathon ONE | G9

---



## SUMÁRIO

|                                                |    |
|------------------------------------------------|----|
| 1. O Projeto .....                             | 2  |
| 2. Diagrama do Pipeline .....                  | 3  |
| 3. Origem dos Dados .....                      | 4  |
| 3.1 Teste da API .....                         | 4  |
| 4. Coleta dos Dados .....                      | 5  |
| 4.1 O Problema .....                           | 5  |
| 4.2 Objetivo .....                             | 5  |
| 4.3 Consumo e Retorno da API .....             | 5  |
| 4.4 Quantidade de Dados e Rotulagem .....      | 6  |
| 4.5 Análise Exploratória .....                 | 6  |
| 4.6 Filtro Textos em Inglês .....              | 7  |
| 4.7 Amostra .....                              | 7  |
| 4.8 Primeiros Insights .....                   | 8  |
| 4.9 Conclusão e Recomendações .....            | 8  |
| 5. Análise e Validação da Amostra .....        | 9  |
| 6. Limpeza e Preparação dos Dados .....        | 12 |
| 6.1 Higienização NLP .....                     | 12 |
| 6.2 Consolidação .....                         | 13 |
| 7. Modelagem .....                             | 14 |
| 7.1 Preparação para o Treinamento .....        | 14 |
| 7.2 Os Modelos Testados .....                  | 14 |
| 7.3 Comparação Final e Escolha do Modelo ..... | 15 |
| 8. Extração de Palavras-Chave .....            | 16 |
| 9. Disponibilização do Modelo .....            | 17 |
| 10. Ideias Futuras .....                       | 19 |
| 11. Conclusão .....                            | 20 |
| Referências .....                              | 20 |



## 1. O PROJETO

O TechMind é uma solução desenvolvida no contexto do Hackathon ONE (grupo G9, time 10), com o objetivo de permitir o enriquecimento e a organização inteligente de conteúdo técnico, facilitando sua classificação, consulta e reutilização.

Profissionais e estudantes de tecnologia consomem diariamente um grande volume de conteúdo técnico (artigos, documentações, tutoriais), o que torna difícil organizar, localizar e reutilizar essas informações posteriormente. O TechMind propõe resolver esse problema recebendo conteúdos técnicos como entrada e utilizando técnicas de Data Science para identificar automaticamente informações relevantes sobre esse conteúdo.

O projeto foi desenvolvido como um MVP (Minimum Viable Product) funcional, com a equipe dividida em dois setores: Data Science (DS) e Desenvolvimento (Back-End e Front-End).

A integração entre o modelo de DS e a aplicação Backend foi realizada via API.

### 1.1 Objetivos da equipe de Data Science:

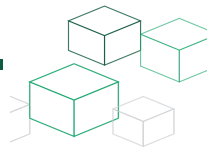
- Obter dados públicos de conteúdo técnico e rotulá-los por categoria, construindo a base de dados própria do projeto;
- Criar e treinar modelos de classificação de conteúdo técnico por categoria;
- Avaliar os modelos treinados e disponibilizar o de melhor performance para consumo via API;
- Realizar a extração de palavras-chave de cada conteúdo analisado;
- Serializar o modelo final, gerando um artefato (.pkl) pronto para ser consumido pelo Back-End.

## 2. DIAGRAMA DO PIPELINE

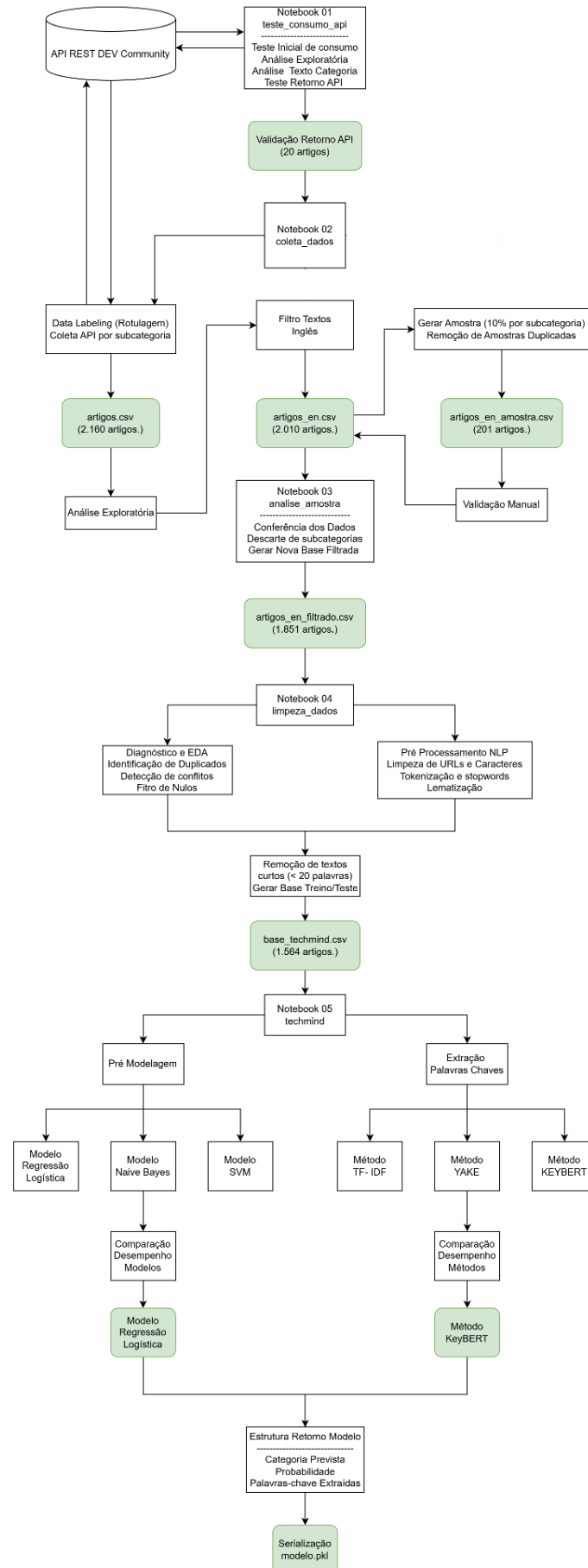
O pipeline de dados do projeto está estruturado em notebooks sequenciais, cada um responsável por uma etapa do processo:

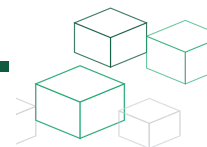
- 01\_teste\_consumo\_api : validação inicial de acesso à API pública do Dev.to
- 02\_coleta\_dados: coleta automatizada dos artigos por subcategoria e rotulagem por categoria
- 03\_analise\_amostra: validação manual da amostra e filtro de subcategorias com baixa aderência
- 04\_limpeza\_dados: limpeza e pré-processamento NLP dos textos
- 05\_techmind: modelagem, extração de palavras-chave e disponibilização do modelo

O diagrama a seguir ilustra as etapas do pipeline de dados do TechMind, desde a coleta dos dados brutos até a disponibilização do modelo para consumo via API.



## Fluxo do Pipeline de Dados Techmind





### 3. ORIGEM DOS DADOS

A base de dados do TechMind foi construída a partir de conteúdo técnico coletado do Dev.to [1], uma comunidade com mais de três milhões de desenvolvedores de software, onde os usuários publicam artigos técnicos, participam de discussões e constroem perfis profissionais.

O site é hospedado pelo FOREM [2], um software open source para construção de comunidades, que disponibiliza em sua documentação uma API pública (v1) [3][4]. Essa API retorna o texto completo dos artigos publicados, junto com as tags associadas a cada um, sem exigir autenticação. A documentação da API não estabelece uma licença específica para os dados disponibilizados.

É importante destacar que os dados retornados pela API não possuem uma categoria pré-definida, as tags indicam apenas o assunto do texto, e a categorização usada no projeto (Backend, Security, Database, Frontend, Cloud, DataScience) foi construída pelo time por meio de um processo de data labeling, conforme detalhado no item 4 (Coleta dos Dados).

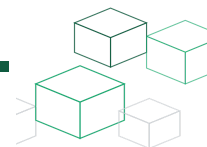
#### 3.1 Teste da API

Antes de iniciar a coleta em maior escala, foi realizado um teste inicial de consumo da API (notebook 01\_teste\_consumo\_api) que retornou uma amostra de 20 artigos, com o objetivo de validar o acesso aos endpoints disponíveis, a estrutura dos dados retornados e a viabilidade de uso dessas informações no projeto. O teste confirmou que:

- os artigos retornados não possuíam dados faltantes nas colunas de texto e tags
- a coluna language, não descrita na documentação oficial, permite filtrar textos em inglês;
- a coluna flare\_tag (tag de destaque) não é viável como categoria, pois a maior parte é nula;
- há colunas que poderiam identificar os autores dos artigos. As mesmas foram ocultadas, por questão de privacidade;
- o parâmetro tags\_exclude não funciona em conjunto com o parâmetro tag, inviabilizando seu uso para descartar artigos de uma tag específica;
- ao consultar uma tag inexistente, a API retorna código 200 com um array vazio, exigindo tratamento específico no código de coleta; já a busca por id retorna 404 quando o artigo não é encontrado, cenário possível quando um artigo é excluído entre a listagem e a busca do detalhe.

Também foram observados pontos sobre o conteúdo dos textos: o campo description é apenas um trecho do texto completo (não um resumo), os artigos incluem não só posts técnicos como também comentários e embeds (links compartilhados), há textos em múltiplos idiomas, e alguns textos contêm emojis, pontos que orientaram as regras de limpeza aplicadas nas etapas seguintes.

O acesso à API não apresentou problemas, e os dados retornados corresponderam ao que estava descrito na documentação oficial. O teste confirmou que a API fornece textos de fato técnicos e utilizáveis para o projeto, além de levantar apontamentos importantes para as próximas etapas: a necessidade de coletar um volume maior de dados e de validar a qualidade dos textos, detalhados no item 4 (Coleta dos Dados).



## 4. COLETA DOS DADOS

### 4.1 O Problema

Para realizar o projeto TechMind é necessário obter textos técnicos da área de tecnologia para treinar um modelo de classificação. O modelo irá classificar os textos pelo assunto principal (categoria). Os dados obtidos via API não possuem um target que represente a categoria, mas os artigos possuem tags associadas que representam o tema. Por isso, foi necessário realizar a rotulagem dos dados (data labeling), gerando o target de cada texto a partir das tags. Também foi extraída uma amostra dos dados para validação manual, a fim de conferir se os textos correspondiam de fato à categoria associada, viabilizando a utilização da base no projeto.

### 4.2 Objetivo

- Realizar a coleta automatizada de artigos técnicos consumindo o endpoint público `/api/articles (v1)`, usando como parâmetro as tags associadas
- Filtrar somente os artigos em inglês
- Realizar a rotulação dos dados, associando um target a cada texto obtido
- Obter uma amostra dos dados para posterior validação manual da qualidade dos textos

A estratégia foi desenhada para capturar dados intencionalmente desbalanceados e com ruídos bi-idioma, simulando um cenário real de engenharia de dados.

### 4.3 Consumo e Retorno da API

Parâmetros de Consulta da API (Endpoint `/api/articles`)

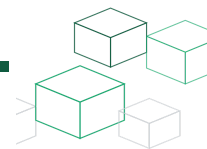
As requisições HTTP do tipo **GET** utilizam os seguintes parâmetros de busca (**Query Parameters**):

- ``tag``: Define a subcategoria técnica alvo da busca (ex: ``python``, ``react``, ``aws``, ``postgres``).
- ``per_page``: Define o limite de artigos retornados por chamada (configurado para ``60`` artigos por subtag).
- ``timeout``: Configurado em ``20s`` para evitar travamentos em caso de oscilação na rede.

#### Dados de Retorno da API (Payload)

A API retorna os dados em formato **JSON**. Para cada artigo, é feita uma chamada em ``/api/articles/{id}`` para compor um dicionário unificado contendo:

- ``categoria``: Rótulo da categoria principal (*target* mapeado, ex: *Frontend*, *DataScience*).
- ``subcategoria``: Subtag específica utilizada na busca.



- ``texto``: Conteúdo completo em Markdown extraído do detalhe (``body_markdown``), essencial para as etapas de NLP.
- ``artigo``: Desempacotamento dos metadados nativos (``id``, ``title``, ``published_at``, ``tag_list`` etc).

A documentação da API informa apenas o retorno 200 para o endpoint de listagem de artigos, sem descrever nenhum código de erro para o caso de a tag passada como parâmetro não existir na base, nesse cenário, a API retorna 200 com um array vazio. Para o endpoint de busca de artigo por id, a documentação informa dois retornos possíveis: 200 para um id válido, e 404 para um id inexistente.

### Tratamento de Erros e Exceções

A coleta foi feita de forma bruta, sem aplicar filtros ou validações neste primeiro momento, independentemente de os dados estarem corretos ou não, com exceção do filtro de artigos em inglês, já que o modelo é treinado nessa língua. Caso os dados se mostrem viáveis, um processo de ETL para tratamento, limpeza e padronização seria realizado posteriormente. Durante a coleta, foram tratados apenas os erros de exceção (falhas na chamada em si, como timeout e/ou instabilidade do servidor); casos de dados inexistentes ou vazios não exigiram tratamento específico, já que os testes iniciais de acesso à API não retornaram esse tipo de erro.

## 4.4 Quantidade de Dados e Rotulagem

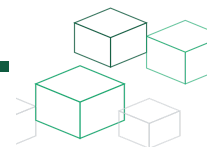
Foram definidas 6 categorias e, para cada uma, 6 subcategorias, utilizadas como parâmetro (tag) no consumo da API:

- Frontend: frontend, javascript, react, css, typescript, nextjs
- Backend: backend, springboot, java, api, php, csharp
- Cloud: cloud, kubernetes, oci, aws, azure, docker
- Database: database, postgres, nosql, sql, mongodb, crud
- Security: security, mfa, cybersecurity, infosec, cryptography, ethicalhacking
- DataScience: datascience, machinelearning, pandas, ai, python, deeplearning

Cada subcategoria foi utilizada como parâmetro (tag) no consumo da API. Os textos retornados foram associados à categoria correspondente, gerando assim o target.

## 4.5 Análise Exploratória

Foram coletados 2.160 registros, distribuídos em 32 colunas. Foi realizada uma análise exploratória inicial, que identificou:



- ausência de dados faltantes nas colunas de texto e tags;
- distribuição perfeitamente balanceada entre as 6 categorias, com 360 registros cada (resultado esperado, já que a coleta buscou 60 artigos por subcategoria);
- colunas que poderiam identificar os autores dos artigos, ocultadas por questão de privacidade;
- predominância de registros em língua inglesa (2.010 de 2.160), com o restante distribuído entre outros idiomas (português, espanhol, alemão, entre outros) ou sem a informação de idioma preenchida.

## 4.6 Filtro Textos em Inglês

Como o modelo seria treinado apenas com textos em inglês, os registros em outros idiomas ou sem o campo *language* preenchido foram desconsiderados. Após esse filtro, a base final ficou com 2.010 artigos em inglês, utilizada como base oficial para a etapa de validação da amostra (item 5).

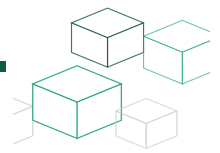
Esse processo, no entanto, impactou de forma desigual as subcategorias, gerando uma leve quebra no balanceamento original:

| Categoria   | Registros |
|-------------|-----------|
| Security    | 346       |
| Cloud       | 345       |
| Frontend    | 341       |
| Backend     | 338       |
| Database    | 324       |
| DataScience | 316       |

## 4.7 Amostra

A partir dos dados coletados foi extraída uma amostra com 10% dos dados, contemplando as 6 categorias e todas as subcategorias associadas, totalizando 204 registros. A maior parte das subcategorias ficou representada com 5 ou 6 artigos; crud e deeplearning ficaram com apenas 4. Após a remoção de registros duplicados (mesmo id e categoria, subcategoria diferente), restaram 201 registros na amostra. A validação e a análise da amostra será detalhada no item 5 desta documentação





## 4.8 Primeiros Insights

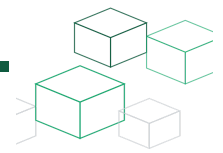
A execução dos scripts de coleta e amostragem resultou em três arquivos CSV:

- artigos.csv (Base Completa) — 2.160 artigos, 360 por categoria, sem colunas identificáveis dos autores
- artigos\_en.csv (Base em Inglês) — 2.010 artigos filtrados por idioma; base oficial de treinamento do modelo
- artigos\_en\_amostra.csv (Base de Auditoria) — 201 artigos sorteados aleatoriamente para validação manual

## 4.9 Conclusão e Recomendações

Nessa fase da coleta foi constatado que a base era provavelmente viável, mas que exigia cuidados antes de alimentar o modelo. Para preparar os textos para o treinamento, foram recomendadas as seguintes ações:

- Remoção de códigos de programação, evitando que termos como `const`, `return` ou chaves `{}` confundam o classificador
- Limpeza de formatação, removendo links, imagens e caracteres especiais de Markdown que atuam como ruído
- Normalização textual, convertendo palavras para minúsculas e aplicando técnicas de Processamento de Linguagem Natural (PLN)



## 5. ANÁLISE E VALIDAÇÃO DA AMOSTRA

Na fase de coleta de dados foi gerado o arquivo `artigos_en.csv`, com 2.010 artigos em inglês. A partir desses dados foi gerado o `artigos_en_amostra.csv`, com 201 artigos sorteados aleatoriamente (cerca de 10% por subcategoria).

A amostra gerada foi validada manualmente. Cada artigo foi lido e avaliado com uma nota:

- 0 = categoria incorreta
- 1 = categoria parcialmente adequada
- 2 = categoria correta

As notas foram registradas em planilha do Google Sheets para categoria, subcategoria e tags do texto, e utilizadas para definir a qualidade dos dados do projeto.

### Avaliação da Amostra

Cada registro da amostra foi avaliado individualmente pelos membros da equipe, para facilitar a validação manual, a amostra foi organizada em uma planilha compartilhada, contendo as principais informações de cada artigo, como **id**, **categoria**, **subcategoria**, **título**, **texto**, **tags**, **url** e **Obs** quando necessário. Cada integrante da equipe identificava seu nome na linha do registro que escolheu validar e a partir da URL realizava a leitura do artigo. Após essa análise, cada integrante registrava as notas de **Texto x Categoria**.

### Avaliação das tags e subcategorias

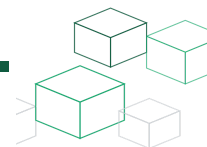
A subcategoria e as tags também foram avaliadas individualmente em relação à categoria atribuída. As subcategorias são utilizadas como parâmetros de busca na API do DEV.to, que utiliza as tags atribuídas pelos autores para localizar e retornar os artigos.

Dessa forma, a avaliação humana permitiu verificar a qualidade dos artigos recuperados por cada busca: caso uma tag apresentasse baixo desempenho ou retornasse conteúdos pouco relacionados à categoria esperada, sua busca poderia ser descartada e substituída por outra subcategoria/tag, buscando recuperar textos mais adequados à categoria utilizada no treinamento.

### Métricas de avaliação

Para medir a qualidade das categorias, foram calculados os percentuais de classificações corretas e de classificações parcialmente adequadas:

- **Nota 2:** quantidade de registros com nota 2 / quantidade total de registros da categoria × 100
- **Notas 1 + 2:** quantidade de registros com nota diferente de 0 / quantidade total de registros da categoria × 100



## Resultados

Os resultados de Texto x Categoria foram comparados entre as seis categorias:

< categoria> (Nota 2 / Nota 1+2)

Backend (78,79% / 93,94%)

Cloud (48,57% / 54,29%)

DataScience (54,84% / 70,97%)

Database (68,75% / 75,00%)

Frontend (68,57% / 77,14%)

Security (85,71% / 97,14%)

No conjunto da amostra, 67,66% dos registros receberam nota 2 e 78,11% receberam nota 1 ou 2.

## Análise das subcategorias

A decisão de manter ou eliminar subcategorias considerou tanto os indicadores quantitativos quanto a análise qualitativa das observações registradas durante a validação. Foram identificados, por exemplo:

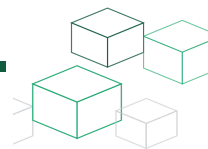
- Cloud/OCI (33,33% de nota 2)
- DataScience/AI (50%)
- DataScience/Deep Learning (0%)
- Frontend/TypeScript (0%)
- Database/MongoDB (60%)

Entretanto, uma baixa porcentagem não foi considerada, isoladamente, motivo para eliminação, pois algumas dessas subcategorias ainda contribuem para trazer textos relevantes e representativos para o treinamento do modelo. No caso de **IA**, por exemplo, foi observado que muitos dos conteúdos retornados tratavam de Inteligência Artificial de forma mais independente de DataScience, portanto, tanto o desempenho percentual quanto a relevância dos registros para a categoria.

## Tratamento dos conflitos

Durante a análise também foram identificados 71 casos em que o mesmo ID e texto estavam associados a categorias diferentes, decorrentes principalmente de artigos que possuíam múltiplas tags. Para resolver esses conflitos de forma padronizada, foram estabelecidas regras hierárquicas:

- Maior nota de desempenho da subcategoria
- Em caso de empate, menor quantidade de registros associados à subcategoria
- Persistindo o empate, menor quantidade de registros associados à categoria

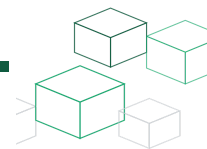


Essas regras permitiram selecionar a classificação com maior evidência de coerência na validação, reduzindo duplicidades e evitando decisões subjetivas.

### Resultado final da amostra

A análise identificou subcategorias com menor aderência entre os artigos recuperados e a categoria atribuída. Como resultado, foram eliminadas as subcategorias **OCI (Cloud)**, **AI (DataScience)** e **DeepLearning (DataScience)**, considerando seu desempenho na validação e a qualidade dos conteúdos recuperados. Após esse filtro, a amostra foi considerada adequada para orientar a seleção das subcategorias utilizadas na coleta, permitindo reduzir buscas com menor aderência e priorizar aquelas com maior capacidade de recuperar conteúdos coerentes com suas respectivas categorias.

Após o descarte das subcategorias com menor desempenho na validação da amostra, restaram 1.851 registros, utilizados como entrada para a etapa de limpeza dos dados.



## 6. LIMPEZA E PREPARAÇÃO DOS DADOS

### 6.1 Higienização NLP

Os dados iniciais foram obtidos através do consumo da API do Dev.to, gerando a base artigos.csv. Em seguida, foi aplicado um filtro para manter somente os artigos em inglês, gerando artigos\_en.csv. A partir dessa base foi extraída uma amostra para validação da classificação dos textos; após essa análise, os registros de subcategorias com baixa aderência entre conteúdo e classificação foram removidos, resultando na base artigos\_en\_filtrado.csv, com 1.851 registros — utilizada como entrada para a etapa de limpeza.

#### Tratamento de Duplicidades e Conflitos

Foram identificados 71 casos de registros com o mesmo id e texto, porém associados a categorias diferentes (conflitos), decorrentes de artigos com até 4 tags associadas. Para resolver esses conflitos, foram aplicadas regras de prioridade definidas com o time, em ordem:

- Manter o registro da subcategoria com maior percentual de nota 2 na validação da amostra
- Em caso de empate, manter o registro da subcategoria com menor quantidade de registros
- Persistindo o empate, manter o registro da categoria com menor quantidade de registros
- Se ainda houver empate, a escolha é aleatória

Além dos conflitos de categoria, foram removidos registros duplicados por id + categoria + texto (mantendo a primeira ocorrência), 3 registros com texto nulo, e registros cujo conteúdo era apenas um placeholder de embed (ex.: {{ \$json.postContent }}), inviáveis para o treinamento.

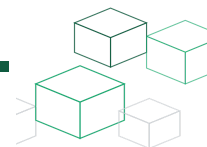
#### Limpeza Estrutural e Pré-processamento NLP

Foi aplicada limpeza de marcações e caracteres não semânticos (front matter, blocos de código, URLs, HTML, emojis) via expressões regulares, seguida de pré-processamento com NLP (spaCy): tokenização, remoção de stopwords e lematização. Após o pré-processamento, a quantidade média de palavras por texto caiu de aproximadamente 1.130 para 610 (redução de quase 45%), resultado consistente com o esperado, já que stopwords costumam representar entre 30% e 45% do volume de um texto em inglês. Foram removidos ainda os textos com menos de 20 palavras após o pré-processamento e 1 registro identificado em outro idioma (japonês).

#### Conclusões

A base final ficou com 1.564 registros, distribuídos em 6 categorias (Backend, Security, Database, Frontend, Cloud e DataScience), prontos para a etapa de modelagem. O processo de limpeza envolveu:

- Remoção de registros nulos e placeholders sem conteúdo real
- Resolução de 82 registros em conflito de categoria (mantendo 71 vencedores, conforme critério de nota por subcategoria)



- Remoção de duplicatas por id, categoria e texto
- Remoção de textos idênticos associados a ids diferentes
- Exclusão de um registro em idioma não inglês
- Limpeza estrutural de marcações residuais (front matter, blocos de código, URLs, HTML, emojis)
- Pré-processamento com NLP (tokenização, remoção de stopwords e lematização via spaCy)

Os dados tratados foram exportados para `base_techmind.csv`, contendo as colunas texto e categoria, utilizadas na etapa de modelagem.

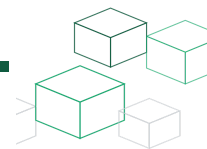
## 6.2. Consolidação

Para possibilitar as etapas de modelagem preditiva e extração de palavras-chave, os dados tratados e higienizados no processo de NLP foram consolidados em um arquivo final unificado, denominado `base_techmind.csv`.

A `base_techmind.csv` reúne os artigos em língua inglesa que passaram pelos critérios de limpeza de ruídos (remoção de elementos Markdown, links, imagens e blocos de código) e normalização textual efetuados na etapa anterior.

O conjunto de dados final contém os atributos essenciais para o treinamento dos algoritmos de aprendizado de máquina, incluindo o texto limpo, o rótulo da categoria técnica (target), a subcategoria mapeada e os identificadores dos artigos.

Este artefato serve como a entrada oficial para a execução dos modelos no notebook `05_techmind`, garantindo a padronização, a vetorização adequada dos dados e a reprodutibilidade dos experimentos.



## 7. MODELAGEM

### 7.1 Preparação para o treinamento

#### Treino e Teste

Antes de treinar os modelos, a base de 1.564 registros foi dividida em treino (80%) e teste (20%), usando amostragem estratificada por categoria [`stratify=categoria`]. Essa escolha preserva a proporção original das seis classes em ambos os conjuntos, evitando distorções causadas pelo desbalanceamento entre categorias (DataScience, por exemplo, tem bem menos registros que Backend).

#### Vetorização

Em seguida, os textos foram convertidos em vetores numéricos com TF-IDF, ajustado apenas no conjunto de treino e aplicado ao teste, para evitar vazamento de informação. O vocabulário foi limitado aos termos mais relevantes [`max_features=5000`] e para considerar palavras isoladas e pares de palavras, capturando mais contexto [`ngram_range=(1,2)`].

Como a remoção de stopwords já havia sido feita na etapa de limpeza (via spaCy), não foi necessário utilizar no TF-IDF.

### 7.2 Os Modelos Testados

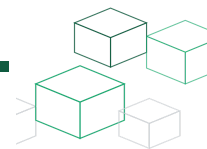
Foram avaliados três algoritmos de classificação, cada um em versão baseline e com ajuste de peso de classes [`class_weight='balanced'`] ou equivalente).

#### Cross Validation

Antes da avaliação no conjunto de teste, cada versão passou por validação cruzada [5-fold, métrica F1-macro], usada para checar a consistência do modelo entre diferentes divisões dos dados de treino. Os três algoritmos apresentaram F1-macro na faixa de 0,79 a 0,81 e desvio padrão baixo entre os folds (0,015 a 0,038), indicando desempenho estável independente da divisão dos dados.

#### Logistic Regression

A validação cruzada apontou F1-macro de 0,7918 no baseline contra 0,8113 no balanced, uma leve vantagem para o balanced nessa etapa. No teste, porém, essa vantagem não se confirmou: o baseline obteve 81% de acurácia contra 80% do balanced. Em DataScience, o balanced elevou o recall (0,68 para 0,73), mas a precision caiu mais (0,89 para 0,71), reduzindo o F1-score da categoria (0,77 para 0,72), um trade-off que não compensou. O baseline foi escolhido como versão final do algoritmo.



## Naive Bayes

O modelo baseline obteve 76% de acurácia, com desempenho mais fraco na categoria DataScience (recall de apenas 0,41). O ajuste feito no modelo balanced [fit\_prior=False] corrigiu esse gargalo, elevando a acurácia geral para 81% e o recall de DataScience de 0,41 para 0,76. Por isso, a versão balanced foi definida como a versão final desse algoritmo.

## SVM

A validação cruzada mostrou F1-macro de 0,7969 no baseline contra 0,8016 no balanced, uma diferença pequena entre as duas versões. No teste, o padrão se repetiu: acurácia idêntica (0,80) e F1-score macro praticamente igual entre as versões, com o balanced trazendo apenas ganhos pontuais em categorias minoritárias (ex: recall de DataScience subindo de 0,70 para 0,76). Como o ganho do balanced foi pontual e não se traduziu em melhora consistente, optou-se por manter o baseline, que preserva a fronteira de decisão original refletindo a distribuição real dos dados de treino, sem impor pesos artificiais às classes.

### 7.3 Comparação final e escolha do modelo

Com as três versões finais definidas, Logistic Regression(Baseline) e Naive Bayes (Balanced) empataram nas métricas gerais (81% de acurácia, F1-macro de 0,81), enquanto o SVM ficou levemente atrás (80%).

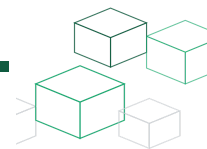
Embora Logistic Regression e Naive Bayes tenham empatado em acurácia e F1-macro, o Logistic Regression leva vantagem em pontos relevantes para a decisão:

- estabilidade entre as classes
- macro avg de precision mais alto (0,82 contra 0,81), indicando maior consistência entre categorias, inclusive as desbalanceadas
- taxa de falsos positivos bem menor na categoria `DataScience` (3 contra 10 do Naive Bayes), historicamente a mais desafiadora entre os modelos testados

Por isso, optou-se pelo modelo Logistic Regression para a implementação final. Como benefício adicional, o predict\_proba do Logistic Regression tende a ser nativamente bem calibrado, o que é útil para a regra de negócio da aplicação (limiar de 0.40 para habilitar revisão manual de palavras-chave).

As matrizes de confusão completas e as demais análises comparativas entre os modelos podem ser consultadas no notebook 05\_techmind.ipynb.





## 8. EXTRAÇÃO DE PALAVRAS-CHAVE

### 8.1 Métodos avaliados

Três abordagens foram testadas para extrair as palavras-chave mais relevantes de cada texto, compondo o campo `informacoes_adicionais` da saída do modelo:

- **TF-IDF:** reaproveita o vetor já usado na classificação, priorizando termos frequentes no texto e raros na base como um todo.
- **YAKE:** método não supervisionado que avalia cada texto isoladamente, sem comparação com outros documentos da base.
- **KeyBERT:** utiliza embeddings semânticos (BERT) para identificar termos que representam o significado do texto como um todo, capturando contexto que os métodos estatísticos não conseguem.

Os três métodos foram testados em uma amostra de 12 textos (2 por categoria) do conjunto de teste, e cada palavra-chave gerada foi classificada manualmente como: **Correta (C)**, **Parcial (P)** ou **Errada (E)** em relação à categoria real do artigo. Nessa primeira rodada de testes, o TF-IDF já ficou atrás dos demais e foi descartado, restando a comparação entre YAKE e KeyBERT.

### 8.2 Texto limpo vs. texto original

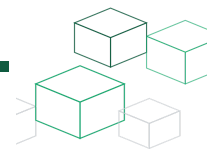
Como o KeyBERT depende de contexto semântico, testou-se também o impacto de aplicar YAKE e KeyBERT sobre o texto original (sem a limpeza), em vez do texto já tratado usado no treinamento do TF-IDF. Essa comparação foi possível porque o índice original de cada artigo foi preservado ao longo do pipeline, permitindo recuperar o texto bruto correspondente a cada registro da amostra.

### 8.3 Resultado e escolha do método

Na avaliação final, com os textos originais, o **KeyBERT** apresentou o melhor desempenho: 58,3% das palavras-chave classificadas como corretas, contra 47,2% do YAKE (21 contra 17 palavras corretas, em uma amostra que gerou 36 avaliações por método). A taxa de erro dos dois métodos ficou próxima (16,7% no KeyBERT contra 13,9% no YAKE), uma diferença de apenas uma palavra errada, ou seja, o ganho do KeyBERT em acertos foi maior que o aumento observado nos erros.

Por isso, o KeyBERT aplicado sobre o texto original foi o método escolhido para a extração de palavras-chave da aplicação. A tabela completa de avaliação palavra por palavra (C/P/E) está disponível no notebook `05_techmind.ipynb`, na seção de Extração de Palavras-Chave.

Vale ressaltar que essa validação foi realizada em uma amostra reduzida, adequada ao escopo de um MVP construído em contexto de hackathon. Em uma aplicação real, essa avaliação seria expandida para uma amostra maior e mais representativa, aumentando a confiabilidade da escolha do método.



## 9. DISPONIBILIZAÇÃO DO MODELO

### 9.1 Estrutura de retorno

Para consumo pela API, foi criada a classe wrapper `ModeloTechMind`, que reúne em um único objeto o modelo de classificação (Logistic Regression), o vetorizador TF-IDF e o modelo KeyBERT. O método `.predict(texto)` da classe executa o fluxo completo: vetorização do texto, classificação, cálculo de probabilidade e extração de palavras-chave, e retorna:

- `categoria`: categoria prevista pelo modelo;
- `probabilidade`: confiança da previsão (probabilidade máxima entre as classes);
- `informacoes_adicionais`: palavras-chave extraídas com KeyBERT.

### 9.2 Serialização do Modelo

A serialização permite salvar os componentes treinados em um arquivo persistente, possibilitando que a aplicação Back-End carregue o modelo pronto para uso, sem necessidade de novo treinamento a cada requisição.

No TechMind, o modelo de classificação (`modelo_lr`), o vetorizador TF-IDF (`vectorizer`) e o modelo KeyBERT (`kw_model`) são reunidos dentro da classe `ModeloTechMind` e serializados com `joblib` em um único artefato: `modelo_techmind.pkl`.

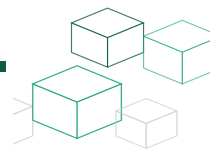
Essa abordagem centraliza em um só arquivo tudo o que é necessário para gerar a previsão completa (categoria, probabilidade e palavras-chave), simplificando a integração com o backend, que só precisa carregar o artefato via `joblib.load` e chamar `.predict(texto)`.

### 9.3 Simulação de uso

Para validar o artefato final, o `.pkl` foi recarregado e testado com textos originais (fora do conjunto de treino), simulando uma requisição real da API. O resultado confirmou que o modelo carregado gera corretamente a saída em formato JSON, com categoria, probabilidade e palavras-chave, mantendo a integração entre os três componentes internos (modelo, vetorizador e KeyBERT). O modelo gerou as seguintes saídas, já no formato JSON esperado pelo backend:

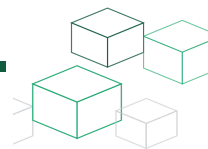
```
[
  {
    "categoria": "Backend",
    "probabilidade": 0.23,
    "informacoes_adicionais": ["failure type", "typescript business", "typescript types"]
  },

```



```
{
  "categoria": "Security",
  "probabilidade": 0.75,
  "informacoes_adicionais": ["quantum encryption", "encryption future", "encryption designed"]
},
{
  "categoria": "Backend",
  "probabilidade": 0.49,
  "informacoes_adicionais": ["risc java", "java risc", "risc processor"]
}
]
```

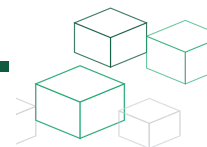
Dos três exemplos, 2 foram classificados corretamente (Security e um de Backend), resultando em 66,7% de acurácia na simulação. O terceiro caso, cuja categoria real era Frontend, foi classificado como Backend com apenas 23% de probabilidade, evidenciando como o threshold de 0,40 atua justamente nesse tipo de situação: uma previsão de baixa confiança que, na prática, seria encaminhada para revisão manual em vez de aceita automaticamente.



## 10. Ideias Futuras

Por ter sido desenvolvido no contexto de um MVP em hackathon, o projeto TechMind possui espaço para evoluções e melhorias. Sugestões para versões futuras:

- **Ampliação da base de dados:** coletar mais registros, principalmente para as categorias com desempenho mais baixo no modelo, como DataScience, e para Backend e Frontend, que apresentaram confusão recorrente entre si nos três algoritmos testados. Avaliar se uma base maior e mais balanceada reduz essa sobreposição e melhora as métricas de classificação.
- **Ajuste de hiperparâmetros:** testar outras combinações de parâmetros nos modelos (ex: regularização no Logistic Regression, kernel e C no SVM, alpha no Naive Bayes), buscando ganhos de performance além dos já obtidos com as versões baseline e balanced.
- **Validação da extração de palavras-chave em maior escala:** expandir a avaliação manual (C/P/E) do KeyBERT para uma amostra maior, dando mais robustez estatística à escolha do método, hoje validada em uma amostra reduzida para o escopo de MVP|Hackathon.
- **Inclusão de outros tipos de conteúdo:** expandir a coleta e o treinamento para além de textos, incorporando outros formatos técnicos (ex: pdf, trechos de código, imagens de diagramas/arquitetura), ampliando o escopo de conteúdo que o TechMind consegue organizar e classificar.



## 11. Conclusão

O pipeline do TechMind partiu da coleta de artigos técnicos via API do Dev.to, passou pela validação da amostra e limpeza dos dados, e chegou a uma base final de 1.564 registros distribuídos em seis categorias (Backend, Security, Database, Frontend, Cloud e DataScience).

Na etapa de modelagem, três algoritmos de classificação foram testados (Logistic Regression, Naive Bayes e SVM), cada um em versão baseline e com ajuste de peso de classes. O **Logistic Regression (baseline)** foi escolhido como modelo final, por apresentar maior consistência entre categorias (macro avg de precision de 0,82) e menor taxa de falsos positivos na categoria historicamente mais desafiadora, DataScience.

Para a extração de palavras-chave, o **KeyBERT** aplicado sobre o texto original se destacou frente a YAKE e TF-IDF, com 58,3% das palavras-chave classificadas como diretamente relacionadas à categoria do artigo.

O modelo final foi reunido em um único artefato serializado (modelo\_techmind.pkl), pronto para ser consumido pelo backend via joblib.load. A simulação de uso confirmou que o artefato retorna corretamente a categoria prevista, a probabilidade da classificação e as palavras-chave, no formato JSON esperado pela API.

## REFERÊNCIAS

- DEV COMMUNITY. Site. Disponível em: <https://dev.to/>. Acesso em: 9 jul. 2026.
- FOREM. Sobre dev.to. Disponível em: <https://github.com/forem/forem/>. Acesso em: 9 jul. 2026.
- API. Versões da API. Disponível em: <https://developers.forem.com/api>. Acesso em: 9 jul. 2026.
- Forem API V1. Documentação da API. Disponível em: <https://developers.forem.com/api/v1>. Acesso em: 9 jul. 2026.